Accelerating Distributed LLM Training on CPUs: A Hybrid Parallelism Approach for Nanochat Across LAN and WAN

This report provides a comprehensive analysis and strategic roadmap for creating a fork of Andrej Karpathy's `nanochat` project to enable distributed training exclusively on Central Processing Units (CPUs) across multiple machines connected via Local Area Networks (LANs) and Wide Area Networks (WANs). The primary objective is to reduce training time through a sophisticated hybrid parallelism approach that combines data and model parallelism. The research addresses the significant architectural challenges involved in adapting a monolithic, educational tool for complex distributed systems, details the technical implementation of a viable parallelism strategy using modern deep learning frameworks, explores critical network optimization techniques, and outlines a practical path forward for development and evaluation. This analysis is grounded entirely in the provided source materials, ensuring all claims are evidence-based.

Architectural Transformation of Nanochat for Distributed Systems

The foundational challenge in achieving the research goal lies in bridging the profound architectural gap between the original `nanochat` project and the requirements of a distributed, CPU-only training environment. `nanochat` was conceived as an educational blueprint, a complete yet compact ChatGPT clone implemented in approximately 8,000 lines of code www.linkedin.com . Its design prioritizes simplicity, reproducibility, and ease of use, enabling users to understand the end-to-end mechanics of building a language model from scratch www.linkedin.com . The project's documentation and implementation are explicitly tied to a GPU-centric paradigm, utilizing high-performance NVIDIA DGX Station hardware equipped with multiple GPUs for its training pipeline www.linkedin.com+1 . It features a simple parameterization scheme, notably a single `--depth` dial to control model complexity, which reflects its focus on a controlled, single-machine experiment rather than a scalable, heterogeneous infrastructure www.linkedin.com . Consequently, any attempt to adapt it for distributed CPU training requires more than just adding flags; it necessitates a fundamental redesign of its core architecture, shifting from a monolithic script to a modular, distributed system where computation, data, and communication are explicitly managed across multiple nodes.

A key aspect of this transformation involves moving away from the project's inherent assumption of a unified computational resource. In the original `nanochat` , the training process operates on a single "cpu" device, meaning that while multiple CPU cores can be utilized, they are not treated as distinct parallel processing units that can be independently scheduled or communicate directly discuss.pytorch.org . The concept of partitioning a large model or dataset across separate machines, each with its own memory space, is alien to the original design. Modern distributed LLM training universally employs complex hybrid parallelism strategies to manage workload and memory footprint, combining data parallelism, tensor parallelism, and pipeline parallelism arxiv.org+2 . The original `nanochat` 's reliance on a single `--depth` parameter does not scale to these dimensions; instead, scaling in a distributed context requires careful consideration of data parallelism degrees, model sharding strategies, and the underlying network topology connecting the workers cse.hkust.edu.hk+1 . Therefore, the first step in the fork is not coding but architectural planning, establishing a new framework for managing distributed state, data flow, and inter-process communication.

Furthermore, the transition to a CPU-only environment introduces different performance characteristics and optimization priorities compared to GPU-based training. While GPUs leverage massive parallelism with hundreds of simple cores, CPUs offer fewer but more powerful cores cccp.eecs.umich.edu . In a distributed setting, efficient utilization of all available CPU cores within and across machines becomes paramount for maximizing throughput. However, since PyTorch models all CPUs as a single device, explicit mechanisms must be devised to parallelize workloads effectively discuss.pytorch.org . This contrasts with GPU training, where the CUDA runtime manages thousands of threads per core. The primary bottleneck in a CPU-based distributed setup shifts from raw floating-point operations per second (FLOPS) to memory bandwidth and network latency. The speed at which gradients and model parameters can be communicated between nodes often becomes the dominant factor in determining total training time, especially over slower WAN links dl.acm.org+1 . This reality places a heavy emphasis on software-level optimizations, particularly those aimed at minimizing the volume of data transferred during synchronization phases. Techniques like gradient compression, sparsification, and intelligent communication scheduling become not just beneficial but essential for achieving the goal of reduced training time arxiv.org+2 .

Finally, the requirement to operate over both LAN and WAN networks adds another layer of complexity. LAN environments typically offer high bandwidth and low latency, whereas WANs are characterized by significantly lower bandwidth and higher, more variable latency arxiv.org+1 . A robust distributed training system must be resilient to these heterogeneous conditions. This means the communication strategy cannot be uniform; it must be aware of network topology and adapt its behavior accordingly. For instance, communication-intensive AllReduce operations might be acceptable within a fast LAN-connected cluster, but they would cripple performance across a WAN. This necessitates the implementation of advanced communication schedulers that can prioritize intra-LAN traffic, delay cross-WAN synchronizations, or employ aggressive data reduction techniques like compression when network conditions are poor arxiv.org+1 . Existing research into geo-distributed ML systems highlights the importance of decoupling communication patterns to handle these disparities, introducing concepts like adaptive compressions based on inter-cluster bandwidth ieeexplore.ieee.... +1 . Thus, the architectural transformation of `nanochat` must embed this network-awareness from the ground up, treating the network not as a transparent utility but as a critical resource whose constraints must be actively managed to achieve optimal performance.

Implementing a Hybrid Data-Model Parallelism Strategy with PyTorch

To effectively reduce training time for a model like `nanochat` in a distributed CPU environment, a hybrid parallelism strategy is essential. The literature on large-scale distributed deep learning overwhelmingly supports the use of hybrid approaches that combine multiple parallelism techniques to balance computational load, memory requirements, and communication overhead arxiv.org+2 . For the specific constraints of this project—CPU-only hardware, multi-machine coordination, and the goal of faster training—the most logical and powerful combination is **Data Parallelism** augmented with **Fully Sharded Data Parallelism (FSDP)**, which represents a form of Model Parallelism. This hybrid approach directly addresses the two primary bottlenecks in large-model training: compute and memory. By distributing the model's parameters, gradients, and optimizer states across all participating nodes, FSDP makes it possible to train models that would otherwise exceed the combined memory capacity of all machines, a critical capability in a CPU-only setting where RAM is a finite and expensive resource docs.pytorch.org+1 .

PyTorch provides the necessary tools to implement this strategy through its native distributed training modules: `DistributedDataParallel` (DDP) and `FullyShardedDataParallel`. DDP is a cornerstone of PyTorch's distributed capabilities, designed to replicate the entire model on each node and then synchronize gradients after each backward pass using an AllReduce operation www.digitalocea... +1. This is a classic example of data parallelism, where each process works on a different subset of the data, but holds a complete copy of the model. While effective, standard DDP can still be memory-intensive, as every worker stores a full replica of the model. FSDP solves this problem by acting as a wrapper around a model or module, automatically sharding its constituent parts—parameters, gradients, and optimizer states—across the data-parallel workers docs.pytorch.org+1. When integrated with DDP, this creates a powerful synergy: DDP handles the high-level inter-node synchronization for backpropagation, while FSDP manages the fine-grained memory sharding within that structure www.linkedin.com+1. This combination is an industry-grade solution for training large models efficiently and is well-suited for CPU environments, as it operates entirely on tensors without requiring specialized hardware offloading discuss.pytorch.... +1.

The technical implementation of this hybrid strategy within the `nanochat` fork follows a clear path. First, the distributed environment must be initialized. This involves setting up SSH access between all machines and installing a CPU-only version of PyTorch (`conda install pytorch torchvision cpuonly -c pytorch`) stackoverflow.co... +1. The training script must be modified to call `torch.distributed.init_process_group`, specifying the `gloo` backend, which is the designated protocol for CPU-to-CPU communication in PyTorch blog.csdn.net+1. Next, the dataset loader must be configured to partition the data correctly among the processes. This is typically achieved using `torch.utils.data.DistributedSampler`, which ensures that each worker process receives a unique, non-overlapping subset of the training data for each epoch zhuanlan.zhihu.com.

Once the environment is set up, the model itself needs to be refactored. Instead of simply wrapping the main model with `DDP`, the recommended pattern is to apply `FullyShardedDataParallel` to the model or its major components (like Transformer blocks) before passing it to `DDP`. This ensures that the sharding logic is applied recursively throughout the model's hierarchy. The training loop also requires adjustments. While the forward pass, loss calculation, and `optimizer.step()` calls remain largely similar to a single-machine setup, the management of gradients is handled internally by FSDP. After `loss.backward()`, FSDP automatically performs the necessary sharded AllReduce operations to aggregate gradients across all workers before applying them to the local shards of the model. The `optimizer.zero_grad()` call should be placed before the forward pass to ensure proper gradient accumulation within the FSDP context arxiv.org. This hybrid DDP+FSDP approach allows the system to scale out across multiple machines, leveraging their combined CPU power and, crucially, their combined RAM to hold larger models, thereby directly addressing the core challenge of training large models on constrained CPU-only hardware.

Feature	Standard Data Parallelism (DDP)	Fully Sharded Data Parallelism (FSDP)	Hybrid DDP+FSDP
Memory Footprint	High; each worker holds a full copy of the model docs.pytorch.org .	Low; model parameters, gradients, and optimizer states are sharded across workers uvadlc-notebooks.r... .	Very Low; combines memory savings of FSDP with inter-node synchronization of DDP docs.pytorch.org .
Primary Use Case	Training models that fit within the memory of a single node but can benefit from multi-GPU/multi-core parallelism arxiv.org .	Training models that are too large to fit in the memory of even a single node docs.pytorch.org .	Training very large models across multiple nodes where each node may have limited memory www.linkedin.com .
Communication	Synchronizes full gradients via AllReduce after each backward pass arxiv.org .	Synchronizes only the gradients corresponding to its local shard of the model parameters arxiv.org .	Combines DDP's AllReduce with FSDP's sharded reductions for a balanced approach docs.pytorch.org .
Backend Requirement	Supports various backends, including NCCL (GPU), TCP, and Gloo (CPU) forums.developer.n... .	Designed to work with any DDP-compatible backend, including Gloo for CPU training blog.csdn.net .	Requires the Gloo backend for all CPU-based communication discuss.pytorch.org .

Optimizing Network Communication for Heterogeneous LAN/WAN Environments

While implementing a hybrid parallelism strategy like DDP+FSDP is crucial for enabling the training of large models on distributed CPUs, it alone is insufficient to guarantee reduced training time, especially when operating over a heterogeneous network of LANs and WANs. In many distributed training scenarios, the network itself becomes the primary bottleneck, with communication overhead consuming a significant portion of the total wall-clock time dl.acm.org+1 . The collective communication primitives that underpin parallelism, most notably the AllReduce operation used to average gradients across all workers, involve substantial data transfer arxiv.org+1 . In a WAN context, where bandwidth is limited and latency is high, these synchronous communication phases can dominate the training loop, leading to periods where all computing resources are idle, waiting for the slowest node to catch up arxiv.org . Therefore, a successful implementation of the `nanochat` fork must incorporate a suite of communication optimization techniques designed to minimize data transfer and intelligently schedule synchronization events to mitigate network congestion and variability.

One of the most impactful optimization strategies is **gradient compression**. This technique aims to reduce the size of the data being transmitted during AllReduce operations without significantly compromising model convergence www.researchgate.... . Simple forms of compression include quantization, such as converting gradients from 32-bit floating-point (FP32) to 16-bit floating-point (FP16) before sending them across the network ojs.aaai.org . More advanced methods involve sparsification, where only a subset of the largest-magnitude gradients are transmitted, while smaller ones are ignored or zeroed out www.mdpi.com+1 . Adaptive compression algorithms represent a further refinement, dynamically adjusting the level of compression based on real-time network conditions, such as available bandwidth, to strike an optimal balance between communication savings and training accuracy ieeexplore.ieee.... +1 . Research has shown that such adaptive schemes can be highly effective in heterogeneous environments, improving training efficiency by tailoring the communication strategy to the specific network topology www.researchga... +1 . For the `nanochat` fork, implementing a baseline compression algorithm, even a simple quantization step, would yield immediate benefits in reducing the volume of data transferred over the network.

Beyond reducing data size, **network-aware scheduling** offers another powerful avenue for optimization. Instead of rigidly enforcing synchronization after every single training step, a scheduler can make intelligent decisions about when to perform AllReduce operations. This can be achieved through several methods. One approach is to batch communications, performing an AllReduce less frequently (e.g., every N steps) and accumulating gradients in the interim, which helps amortize the fixed cost of initiating a collective operation ojs.aaai.org . Another, more sophisticated approach is to overlap communication with computation. Frameworks like iBatch propose novel communication approaches that aim to batch parameter communication and forward computation to overlap them, ensuring that while some nodes are calculating gradients, others are already transmitting their results ojs.aaai.org . For geo-distributed settings, researchers have proposed adaptive schedulers like NetStorm, which uses heuristics to accelerate parameter synchronization by optimizing the order and timing of communication flows across data centers arxiv.org+2 . For the `nanochat` project, a simpler heuristic could be implemented: grouping machines by their IP address ranges to identify which nodes are on the same local network segment and prioritizing direct communication between them, while queuing or delaying communications between different segments.

Finally, understanding and leveraging **topology-aware communication** is vital for maximizing efficiency. The physical layout of the network—how machines are connected to switches and routers—affects communication latency and bandwidth www.sciencedirect.... . An ideal communication pattern would preferentially route traffic along the fastest available paths, such as within the same rack or switch, while avoiding congested links or traversing slow WAN connections unnecessarily developer.nvidia.co... . Some advanced systems co-optimize the parallelization strategy with the network topology to minimize interference between communication flows cse.hkust.edu.hk . While implementing such a system from scratch is complex, the `nanochat` fork can adopt a simplified version of this principle. By allowing the user to specify a network topology file or by automatically inferring it from host connectivity, the training script could assign roles to workers in a way that clusters communication-heavy tasks together on the same fast LAN segment. This reduces the need for data to traverse the slow WAN link for every synchronization step, reserving such costly operations for less frequent, bulk updates. By integrating these communication optimization techniques, the forked `nanochat` can move beyond a naive parallel implementation and build a system that is genuinely optimized for performance in a real-world, heterogeneous network environment.

Performance Analysis and Scalability Considerations for CPU-Only Clusters

Evaluating the performance and scalability of the distributed `nanochat` fork requires a nuanced understanding of the trade-offs inherent in distributed CPU training. The primary metric of interest is the reduction in wall-clock training time, which is influenced by a complex interplay of computational throughput, memory capacity, and, most critically, network efficiency. While the hybrid DDP+FSDP strategy enables the training of larger models by leveraging the aggregated RAM of multiple machines, the actual speedup realized is contingent upon how effectively the system can hide communication latency behind computation and avoid stragglers in the network arxiv.org . The performance landscape is further complicated by the inherent heterogeneity of typical multi-machine setups, where nodes may differ in CPU speed, number of cores, and RAM capacity, leading to imbalanced workloads if not managed carefully www.sciopen.com+1 .

A significant performance bottleneck in distributed training is the AllReduce operation, which synchronizes gradients across all participating nodes arxiv.org+1 . On a LAN with high-bandwidth, low-latency connections, this can be relatively efficient. However, on a WAN, the limited bandwidth and higher latency can cause the AllReduce phase to take a disproportionately long time, leading to significant idling of the CPU cores on other machines arxiv.org . This phenomenon, known as the "straggler problem," can severely degrade the overall speedup, making the n-node training time much worse than the single-node time divided by n. The effectiveness of communication optimization techniques like gradient compression and scheduling becomes paramount here. Aggressive compression can drastically cut down the amount of data that needs to be moved, while intelligent scheduling can help overlap communication with computationally intensive forward and backward passes, keeping the CPUs busier ojs.aaai.org+1 . However, there is a delicate trade-off: excessive compression can introduce noise into the gradients, potentially slowing down model convergence or degrading final accuracy, a risk that must be empirically evaluated for the specific model and task ieeexplore.ieee.... +1 .

Scalability in a distributed system refers not only to the ability to add more machines to reduce training time but also to the stability and efficiency of the system as the number of participants grows. As the number of workers increases, the complexity of managing the communication graph and the total volume of data exchanged also increase. Collective communication algorithms like AllReduce do not scale linearly; their performance can degrade due to increased contention on network switches and routers www.researchgate.... . Therefore, while adding more machines generally increases total computational power, the law of diminishing returns often applies to communication efficiency. The success of scaling the `nanochat` fork will depend on its ability to maintain a favorable ratio of computation time to communication time. Benchmarking against a single-machine baseline is a critical first step. The forked version should be tested on a small LAN cluster to establish a baseline performance improvement and validate the correctness of the parallelism and sharding logic. Key metrics to track include per-iteration wall-clock time, throughput in tokens/second, and model accuracy on a validation set dl.acm.org+1 .

Subsequent testing should focus on the WAN scenario to stress-test the communication optimization strategies. The performance drop when switching from LAN to WAN will highlight the effectiveness of techniques like gradient compression. Further investigation is needed to understand the impact of hardware heterogeneity. If one machine in the cluster is significantly slower or has less RAM, it can

become a bottleneck, slowing down the entire training process. Heterogeneity-aware frameworks exist that aim to optimize resource allocation and task scheduling to account for these differences, minimizing total training latency by better matching tasks to available resources arxiv.org+1 . For the `nanochat` fork, a basic approach could involve running benchmarks on the target machines beforehand to gather performance profiles and then assigning larger data partitions or more complex computational tasks to the faster machines. Ultimately, the scalability of the system will be determined through empirical measurement, characterizing how training time and efficiency change as the number of machines, the complexity of the model, and the quality of the network vary. This empirical analysis is essential for validating the project's core objective and providing actionable insights for future development.

Factor	Impact on Performance & Scalability	Mitigation Strategies
Network Bandwidth/Latency	Primary bottleneck in WAN settings. Low bandwidth increases AllReduce time, while high latency increases idle time. arxiv.org+1	Gradient Compression, Adaptive Scheduling, Topology-Aware Routing. arxiv.org+2
Model Size / Memory	Large models exceed single-machine RAM. FSDP enables training but increases communication volume. dl.acm.org+1	FSDP for memory reduction, Gradient Sparsification. arxiv.org+1
Hardware Heterogeneity	Stragglers and imbalanced workloads reduce overall cluster efficiency. www.sciopen.com+1	Heterogeneity-aware scheduling, dynamic load balancing. arxiv.org+1
Number of Workers	Increased computation power, but communication overhead and contention grow super-linearly. www.researchgate....	Efficient collective algorithms, hierarchical All-Reduce. www.researchga... +1
Gradient Compression	Reduces communication volume but risks impacting model convergence and final accuracy. ieeexplore.ieee.... +1	Adaptive compression rates, error feedback mechanisms, thorough accuracy validation. arxiv.org+1

Practical Implementation Roadmap and Technology Stack

To successfully execute the research goal, a pragmatic and structured implementation roadmap is required, centered on a carefully selected technology stack. Based on the analysis, PyTorch emerges as the most suitable framework due to its native, low-level support for the necessary distributed primitives, extensive documentation, and strong community backing for DDP and FSDP docs.pytorch.org+1 . While alternative frameworks like Ray offer powerful abstractions for scaling AI workloads, they may introduce unnecessary complexity for this specific task of modifying a single project, making PyTorch's granular control a more direct and effective choice www.linkedin.com+1 . The following roadmap outlines a phased approach, starting with a functional baseline and progressively incorporating advanced features.

Phase 1: Environment Setup and Baseline DDP Implementation
The initial phase focuses on establishing a working distributed environment and implementing a basic data-parallel baseline.

1. **Cluster Configuration:** Set up a cluster of machines, ensuring they can communicate via SSH without password prompts. Install a CPU-only version of PyTorch and its dependencies on all nodes stackoverflow.co... +1 .
2. **DDP Initialization:** Modify the `nanochat` training script to initialize the PyTorch distributed environment using `torch.distributed.init_process_group`. Crucially, this must be configured with the `gloo` backend to facilitate CPU-to-CPU communication blog.csdn.net+1 .
3. **Data Partitioning:** Integrate `torch.utils.data.DistributedSampler` into the DataLoader. This ensures that each worker process receives a unique, non-overlapping portion of the training dataset, which is fundamental for correct data-parallel execution zhuanlan.zhihu.com .
4. **Loss Reduction:** Wrap the model with `torch.nn.parallel.DistributedDataParallel` and ensure that the final loss is properly synchronized across all processes before being logged or used for the backward pass. This establishes a functional, albeit memory-intensive, distributed training setup docs.pytorch.org .

Phase 2: Integration of Fully Sharded Data Parallelism (FSDP)
This phase introduces model parallelism to overcome memory limitations.

1. **Refactor Model Definition:** Identify the core model components in `nanochat` (e.g., the Transformer architecture). Refactor the code to wrap these components with `torch.nn.parallel.FullyShardedDataParallel`.
2. **Integrate with DDP:** Apply FSDP to the model *before* wrapping it with DDP, following the recommended hybrid pattern for optimal performance www.linkedin.com+1 . This creates a sharded model that is replicated across data-parallel workers.
3. **Training Loop Adjustment:** Update the training loop to accommodate FSDP's internal gradient handling. Specifically, place `optimizer.zero_grad()` before the forward pass to align with FSDP's expected workflow arxiv.org . Test this configuration on a small-scale problem to confirm that the model fits within the combined memory of the cluster and trains correctly.

Phase 3: Development of Communication Optimizations
With a stable parallel training implementation, this phase focuses on mitigating network bottlenecks, especially for WAN deployment.

1. **Implement Gradient Compression:** Start with a simple yet effective technique. Modify the training step to convert gradients to a lower-precision format, such as `torch.float16`, before the AllReduce operation. This immediately reduces the communication payload.
2. **Explore Advanced Techniques:** Investigate more sophisticated methods like gradient sparsification (sending only top-k gradients) or adaptive compression algorithms that adjust based on measured network conditions www.researchga... +1 .
3. **Develop a Basic Scheduler:** Implement a rudimentary communication scheduler. For example, modify the training loop to perform AllReduce operations every N steps instead of every step to amortize communication costs ojs.aaai.org . For WAN testing, this can be enhanced to trigger synchronizations only when network traffic is expected to be low.

Phase 4: Rigorous Testing and Benchmarking
The final phase involves comprehensive evaluation to validate performance gains and ensure model integrity.

1. **LAN Benchmarking:** Conduct initial tests on a fast LAN-connected cluster. Measure and compare wall-clock time, throughput (tokens/second), and memory usage against the single-machine baseline. Monitor model accuracy to ensure that communication optimizations have not degraded performance.
2. **WAN Simulation and Deployment:** Deploy the fork on machines connected via a simulated or real WAN link. Quantify the performance degradation and evaluate the effectiveness of the implemented communication optimizations in this challenging environment.
3. **Heterogeneity Testing:** If possible, test the system on a cluster with heterogeneous hardware to observe the impact of stragglers and assess the robustness of the implementation.

By following this systematic roadmap, the developer can incrementally build a robust and efficient distributed training system for `nanochat`, transforming it from a simple educational tool into a powerful demonstration of scalable, resource-conscious AI training.